

durable-run

scripts/lib/durable-run.sh — durable remote-execution helper

Revision: 1 **Last modified:** 2026-08-15T00:00:00Z **Status:** active
Item: BOB-064 (Lava P1 port — durable remote execution via systemd-linger)

Overview

Source library exposing a small API for launching long-running jobs on a remote (or local) host such that they **survive the SSH / login session ending**. Backed by `loginctl enable-linger + systemd-run --user --unit=<name> --collect`, following the pattern documented in `docs/PORTING-FROM-LAVA.md` §P1 and ported verbatim (with Boba-idiomatic docs + first-boot linger hint) from `../lava/submodules/containers/scripts/lib/durable-run.sh`.

Prerequisites

- `bash`, `systemctl` (`--user`), `systemd-run` (`--user`), `loginctl`, `awk`, `cat`, `mkdir`, `sleep` — all standard on any systemd Linux host.
- A running `systemd --user` manager (verify with `systemctl --user is-system-running`; state `running` or `degraded` is fine, anything else means no user manager).
- `loginctl enable-linger` performed **ONCE per user per host** by an operator with `root` — see [First-boot](#). Without `linger` the user manager is torn down at the last logout and the launched `.service` unit dies with it.

The helper is **rootless in normal operation** — no `sudo`, no interactive prompt (§11.4.161 `rootless` + §11.4.234 `always-unblocked`).

Usage examples

Fire-and-forget an inline command, poll for completion, fetch log

```
source scripts/lib/durable-run.sh

UNIT="boba-qa-nightly-$(date +%s)"

# Launch a real long-running command – this returns as soon as
# the .service
# is registered; the actual body runs under the user systemd
# manager.
durable_launch_cmd "$UNIT" \
    'bash scripts/pre_build_verification.sh && bash challenges/
    scripts/run_all_challenges.sh'

# Optional: drop the SSH session here. The unit keeps running.

# Later (same host, new session), block for completion (max 2h)
# and read rc:
rc="$(durable_wait_sentinel "$UNIT" 7200)"
echo "job exit code: $rc"

# Print the captured stdout+stderr:
durable_fetch_log "$UNIT"

# Reap artifacts:
durable_stop "$UNIT"
```

Launch an existing runner script instead of an inline command

```
durable_launch my-deploy ./scripts/deploy-remote.sh
```

Check status mid-flight

```
if durable_is_active my-deploy; then
    echo "still running as PID $(durable_main_pid my-deploy)"
fi
```

API reference

Function	Args	Returns
durable_launch	<unit> <script_path>	0 on registered, non-0 on systemd- run failure

Function	Args	Returns
<code>durable_launch_cmd</code>	<code><unit> <command...></code>	0 on registered, non-0 on systemd-run failure
<code>durable_is_active</code>	<code><unit></code>	exit 0 iff the unit is active
<code>durable_main_pid</code>	<code><unit></code>	echoes MainPID (0 if none)
<code>durable_wait_sentinel</code>	<code><unit> [timeout_s]</code>	echoes captured exit code; returns 1 on timeout
<code>durable_fetch_log</code>	<code><unit></code>	cats combined stdout+stderr from <code>\$DURABLE_DIR/<unit>.log</code>
<code>durable_stop</code>	<code><unit></code>	stops the unit, reset-fails it, removes runner/log/sentinel

All functions accept the unit name with or without the `.service` suffix.

Environment

- `DURABLE_DIR` — where the runner script, combined log, and sentinel file live per unit. Defaults to `$XDG_CACHE_HOME/remoteexec` or `~/.cache/remoteexec`. Overridable per-invocation.
- `XDG_RUNTIME_DIR` — auto-derived from `id -u` if unset (needed for `systemctl --user` to talk to the user manager over its bus socket).

Edge cases

- **No user systemd manager** (containers without one, `pid1-init` hosts, some CI runners): `systemctl --user is-system-running` returns `offline` or an error. Callers **MUST** detect this and `SKIP-with-reason` (§11.4.3). The helper functions do **NOT** self-skip — they fail loudly, which is the correct behaviour when durability is expected.
- **Linger not enabled**: the helper prints a one-line reminder on source, but does **NOT** self-escalate. The unit will still **LAUNCH** under the user manager; it will **DIE** with the last logout because the user manager itself exits.
- **Unit-name collision**: `systemctl --user reset-failed` is called before every launch to clear a prior failed state; a still-active same-named unit will cause `systemd-run` to fail with a clear

error — call `durable_stop` first or pick a unique name (recommend timestamp / `$$-RANDOM` suffix).

- **The tail -N trap:** never pipe the durable job's stdout through `tail -N` — pipes buffer until the writer exits. Read `<unit>.log` directly (or `tail -f` on the file path) instead.

First-boot / sudo-free (§11.4.234)

`loginctl enable-linger` normally requires root. It is a **one-time per user per host** operation. On a fresh host:

```
sudo loginctl enable-linger "$USER"
# verify:
loginctl show-user "$USER" -p Linger --value    # → "yes"
```

The helper prints a NOTE: to stderr on source when it detects `Linger=no`, so the operator sees the required action but the mechanism is never blocked (§11.4.234 always-unblocked).

Internal behaviour

1. `durable_launch_cmd` writes a wrapper `<unit>.runner.sh` under `$DURABLE_DIR` that redirects the body's stdout+stderr into `<unit>.log`, captures the body's exit code, and writes it to `<unit>.COMPLETE` (the sentinel) — ALWAYS, in both success and failure paths.
2. `loginctl enable-linger` is invoked (idempotent, no-op if already enabled or if the caller is non-root).
3. `systemctl --user reset-failed <unit>.service` clears any prior failed state so re-launches under the same name succeed.
4. `systemd-run --user --unit=<name> --collect bash <runner>` registers the transient user .service unit and returns immediately; the body executes asynchronously under the user manager.
5. Consumers poll `durable_is_active` / `durable_wait_sentinel` / `durable_fetch_log` at their own cadence.

Verification

Anti-bluff regression guard: `challenges/scripts/durable_run_helper_challenge.sh` (BOB-064 §11.4.115 RED/GREEN polarity):

- Launches a real sleeper via `durable_launch_cmd`.
- Reads the unit's MainPID from `systemctl --user show`.
- Reads `/proc/<pid>/cgroup` to prove the job runs in an independently-managed .service cgroup — DIFFERENT from the launching shell's session scope. This is the runtime-

signature (§11.4.108) that proves the fix is active: a session-scope cgroup would be reaped with the login session (the pre-fix failure mode).

- Waits on the sentinel, asserts exit code 0, asserts both log markers landed.

The RED polarity (default RED_MODE=1) inverts the assertions so the same challenge PASSes on a broken tree and FAILs on a working one — proving the guard genuinely catches regressions per §11.4.115(F).

Honest SKIP-with-reason (§11.4.3): when no systemd --user manager is running (container, CI, non-systemd host), the GREEN half exits 0 with a SKIP: prefix. Durability is impossible-by-topology there.

Related scripts

- scripts/deploy-remote.sh — remote deploy driver; a natural consumer of durable_launch when deploying to a host over SSH (wire on demand per §11.4.197 follow-up).
- challenges/scripts/run_all_challenges.sh — challenge aggregator; can wrap its full sweep in durable_launch_cmd when the sweep runs on a remote host (wire on demand).

Last verified

2026-08-15 — helper ported from Lava, challenge GREEN under real systemd --user manager on a systemd-linger-enabled host; captured evidence:

```
job cgroup = /user.slice/user-1000.slice/user@1000.service/
app.slice/boba-durable-guard-*.service
self cgroup = /user.slice/user-1000.slice/user@1000.service/
app.slice/tmx-boba-*.scope
PASS: durable-run.sh helper — job survived launcher in
own .service cgroup, ran to completion (BOB-064 fix intact)
```